

Documento de Avance: Pruebas

Proyecto: VAMOS

Revisión: [1.0]

[22 de Junio de 2026]

Integrantes: Freddy Galarza

Pablo Mery

Luis Nuñez

Profesor: Ignacio Gonzalez

Contenido

Introducción	3
Resumen y Contexto	3
Fase de Pruebas	4
3.1 Plan de Pruebas.....	4
3.2 Entorno y Base de Datos de Pruebas	4
3.3 Evidencias.....	5
3.4 Mejoras	5
Conclusiones y Lecciones Aprendidas	5
Consideraciones futuras.....	6

Introducción

Este documento detalla el Estado de Avance 3 del proyecto "VAMOS". Su objetivo principal es detallar y evidenciar la ejecución de las pruebas funcionales realizadas sobre los componentes del software, garantizando la trazabilidad entre los requisitos iniciales y los resultados obtenidos. Adicionalmente, se documentan las mejoras implementadas a raíz de estas pruebas para asegurar la calidad, tolerancia a fallos y concurrencia del sistema.

Resumen y Contexto

Desde la formulación inicial del proyecto, VAMOS ha evolucionado de un concepto teórico a un producto terminado. Durante este proceso, se tomaron decisiones estratégicas para optimizar recursos, garantizar la escalabilidad y asegurar el cumplimiento:

1. **Pivoteo Estratégico del Modelo de Negocio:** Inicialmente, se planteó abarcar la totalidad de eventos de toda la ciudad de Santiago. Sin embargo, para asegurar un producto final de alta calidad y datos verificables ahora concebimos a las municipalidades como nuestros clientes primarios. La versión comercial actual valida esta viabilidad utilizando a la Municipalidad de Providencia como socio estratégico y fuente centralizada de datos.
2. **Optimización Arquitectónica (De Serverless a Contenedores):** El diseño original contemplaba una infraestructura altamente fragmentada (uso de AWS Lambda, Amazon RDS y múltiples instancias). Se decidió refactorizar hacia un Monolito Modular orquestado con Docker (docker-compose) alojado en una única instancia EC2 en una VPC pública.
 - *Justificación:* El uso de funciones Serverless (Lambda) para tareas programadas generaba un sobre costo y complejidad innecesaria cuando ya se disponía de un servidor activo. Se reemplazó exitosamente por un Cron Job nativo de Linux que gatilla los scripts de ingesta. Asimismo, la base de datos se alojó dentro del mismo clúster de contenedores en lugar de RDS para maximizar el aprovechamiento de la capa gratuita (*Free Tier*) de AWS.
3. **Implementación de Inteligencia Artificial Avanzada (pgvector):** Aunque Vambot siempre estuvo contemplado, la arquitectura de datos evolucionó drásticamente. Se descartó una búsqueda relacional clásica en favor del despliegue de PostgreSQL con la extensión pgvector. Esto permitió integrar un Worker asíncrono (FastAPI) que genera *embeddings* matemáticos en segundo plano solo cuando detecta nuevos eventos, logrando búsquedas semánticas ultrarrápidas y precisas.
4. **Refactorización de Autenticación y Deep Linking:**
 - Se delegó la responsabilidad de la autenticación inicial al flujo Google OAuth 2.0. Sin embargo, dado que este proveedor no proporciona la edad en su token inicial, el sistema delega la validación de mayoría de edad a una interfaz posterior obligatoria dentro de la app móvil, donde el usuario debe ingresar manualmente su fecha de nacimiento. Tras la verificación, se emiten tokens JWT propios desde Django para el control de sesiones.

- Se descartó la dependencia de terceros (Branch.io) para las invitaciones a grupos logísticos debido a restricciones de sus planes gratuitos, en su lugar se implementó una solución nativa (Share.share) y modales de ingreso de códigos, manteniendo la usabilidad sin costos adicionales.

Fase de Pruebas

3.1 Plan de Pruebas

El plan de pruebas se diseñó para validar integralmente los módulos de la aplicación. A continuación, se detallan los casos de prueba ejecutados y aprobados:

- CP-01: Autenticación OAuth y Restricción de Edad. Se validó que el login con Google funcione correctamente y que el sistema bloquee la creación de perfiles para usuarios menores de 18 años. (Resultado: Exitoso)
- CP-02: Obtención Dinámica y Renderizado. Se verificó que el "Data Seeder" extraiga la cartelera de Providencia mediante web scraping, guarde los datos (Upsert) en PostgreSQL, y los marcadores se rendericen en el mapa. (Resultado: Exitoso)
- CP-03: Asistencia Inteligente (Vambot). Se probó el microservicio FastAPI consumiendo la API de Gemini, comprobando que responde a prompts usando exclusivamente el contexto de los eventos locales de la BD. (Resultado: Exitoso)
- CP-04: Gestión Logística y Compartición. Se validó la creación de un grupo privado y el uso del intent nativo del sistema operativo (Share) para enviar códigos de invitación vía WhatsApp/Instagram. (Resultado: Exitoso)
- CP-05: Sincronización Concurrente de Estados. Se confirmó que los cambios de estado ("En camino", "Llegué") de un usuario se persistan y visualicen concurrentemente para los otros miembros del grupo. (Resultado: Exitoso)
- CP-06: Ruteo Nativo (Google Maps). Se validó la integración con la Directions API para trazar la ruta desde la ubicación del usuario hasta el evento seleccionado. (Resultado: Exitoso)

3.2 Entorno y Base de Datos de Pruebas

Las pruebas se ejecutaron en el entorno Cloud desplegado en AWS EC2, replicando las condiciones de producción:

- Base de Datos: PostgreSQL con la extensión pgvector (requerida por el módulo de IA), alojada en contenedores Docker.
- Poblado de Datos (Data Seeding): Se utilizaron datos reales extraídos automatizadamente, no data sintética insertada a mano.

- Usuarios Aislados: Para auditar los módulos logísticos y de sincronización concurrente (RF-04 y RF-05), se generaron cuentas de prueba asépticas aisladas para prevenir colisiones de sesión y garantizar la pureza de las pruebas transaccionales.

3.3 Evidencias

Las evidencias de los resultados de los planes de prueba han sido adjuntadas en el documento Plan de Pruebas Funcionales VamosApp.

3.4 Mejoras

En respuesta a los resultados de las pruebas de estrés y los ciclos de QA, se implementaron mejoras críticas para la estabilidad y completitud del sistema:

- Mitigación de OOM (Out Of Memory) en AWS: Se detectaron caídas abruptas en la capa gratuita de EC2 debido a la saturación de RAM por parte de los contenedores Docker. Se resolvió configurando un archivo de memoria virtual temporal (SWAP) en Linux, garantizando la disponibilidad de la API.
- Refactorización Dinámica: El script original de extracción requería cambios manuales para las URLs. Se refactorizó en Python para que identifique y procese dinámicamente el mes en curso, previniendo fallos en la ingesta de datos a futuro.
- Ambientes Aislados para Concurrencia: Se implementó una política estricta de testeo con cuentas aisladas para comprobar la inexistencia de fugas de información o cruce de sesiones HTTP.

Conclusiones y Lecciones Aprendidas

El plan de pruebas ejecutado validó exitosamente todos los requerimientos funcionales y lógicos de VAMOS. La aplicación es capaz de generar su contenido, administrar la autenticación segura, aplicar reglas de negocio (restricción >18 años), asistir mediante Inteligencia Artificial y manejar la concurrencia de sesiones logísticas.

Lecciones Aprendidas:

La orquestación de contenedores en instancias con recursos limitados (Free Tier de AWS) exige una planificación minuciosa del uso de la memoria. La implementación de memoria SWAP fue vital para evitar tiempos de inactividad.

Las pruebas exhaustivas con cuentas aisladas son innegociables al desarrollar sistemas concurrentes (grupos y estados logísticos), ya que permiten detectar fallos de sobrescritura de estados que no son evidentes en pruebas unitarias simples.

Consideraciones futuras

Durante el periodo de validación y observación, se recogió feedback de los usuarios de prueba. Si bien los requerimientos iniciales se han cumplido en su totalidad, para futuras iteraciones del proyecto se ha documentado la posibilidad de incluir:

- Subida de fotografías: Permitir a los usuarios compartir imágenes dentro del grupo de los eventos a los que asisten.
- Chat Interno Logístico: Una sala de mensajería en tiempo real dentro del "grupo de evento" para coordinaciones rápidas sin depender de aplicaciones externas (WhatsApp).